

MATERIAL EXTRA

Tipagem estática



growdev

TIPAGEM ESTÁTICA

No Typescript podemos trabalhar com a tipagem dinâmica que já é característica do Javascript. Mas uma das principais features do superset é a possibilidade de definirmos **tipos estáticos** para variáveis, parâmetros e retornos de funções e métodos.

A **tipagem estática** oferece uma série de vantagens aos programadores. Podemos criar elementos com **regras** bem definidas quanto ao tipo de dados, evitando que dados com outros formatos sejam aceitos. Como consequência, a probabilidade de bugs na construção e na manutenção dos elementos diminui.

Com mais **restrições de tipagem**, maior é o **controle** do programador sobre o código feito. A primeira vista, é normal ao programador Javascript pensar que o Typescript apenas adiciona mais trabalho devido a definição dos tipos, porém todas as vantagens superam essa situação.

Os principais tipos disponíveis em Typescript são listados abaixo:

Tipo	Descrição
boolean	Valores booleanos (true ou false)
string	Caracteres e textos
number	Números (inteiros ou decimais)
null	Valor nulo (usado para representar uma ausência intencional de valor)
undefined	Valor nulo (quando não há valor inicializado ou retornado)
any	Qualquer tipo
void	Valor vazio (usado para tipar funções que não retornam valor)

Os tipos que o Typescript oferece em sua maioria são todos aqueles que as linguagens de programação tipadas também possuem, como `string`, `number` e `boolean`. Ainda, um importante tipo é o `any` que representa qualquer valor. Na prática, o `any` define um **tipo dinâmico** assim como o Javascript.

Quando a **ausência de valor**, podemos notar duas alternativas: o `null` e o `undefined`. Ambos possuem o mesmo objetivo que é definir um valor nulo, mas semanticamente possuem diferenças. O `null` define uma ausência de valor **intencional** – queremos de fato mostrar que o dado é nulo –, enquanto o `undefined` pode ser obtido por variáveis criadas e ainda **não inicializadas**.

Para criarmos variáveis usando a tipagem estática do Typescript, basta adicionar o respectivo tipo após o nome da variável em sua criação separado pelo caractere `:` (dois pontos).

Exemplo usando Typescript:

```
// Tipagem estática (index.ts)
let nome: string = "Growdev";
nome = 12345; // ERRO
```

Mesmo exemplo usando Javascript:

```
// Tipagem dinâmica (index.js)
var nome = "Growdev";
nome = 12345; // OK
```

No exemplo acima, criamos uma variável chamada nome que recebe um texto "Growdev". Usando o Javascript, não definimos o tipo da variável e por isso a atribuição de um valor numérico na próxima linha está OK e irá funcionar. Já o exemplo com Typescript adicionou a tipagem estática string, sendo que a atribuição de um número gera erro de sintaxe.

```
let nome: string

O tipo 'number' não pode ser atribuído ao tipo 'string'. ts(2322)
Exibir o Problema  Nenhuma correção rápida disponível

nome = 12345; // ERRO
```

O Typescript também realiza uma **auto tipagem** (ou tipagem implícita) quando uma variável é inicializada no momento de sua criação. Ou seja, a variável recebe o tipo de acordo com o valor recebido. Caso uma variável sem tipo seja declarada e não inicializada, seu tipo é inferido é "any".

```
let nome = "Growdev"; // type: string
let naoInicializada; // type: any
```

Definir um tipo para uma variável pode auxiliar na descoberta das funcionalidades disponíveis sobre o dado. Se uma variável do tipo string for criada, o Typescript irá sugerir operações de string através do VSCode.

```
let texto: string = "Growdev";  
texto.
```

- length
- localeCompare
- match
- matchAll
- normalize
- padEnd
- padStart
- repeat
- replace
- search
- slice
- split

OPERAÇÕES

As operações existentes no Javascript funcionam da mesma forma no Typescript. A vantagem é ter a chance de saber quais os tipos de dados estão sendo usados, evitando possíveis surpresas.

```
// Operador aritmético  
let x: number = 10;  
let y: number = 20;  
let soma = x + y; // type: number  
  
// Operador lógico  
let condicao: boolean = false;  
console.log(condicao == false); // resultado: true  
  
// Condicionais  
if(x > 10) {  
    console.log("x é maior do que 10");  
}
```

```
// Repetição
let texto: string = "abc";

for (const item of texto) {
  console.log(item); // Log: a b c
}
```

FUNÇÕES

No contexto das **funções** e métodos de classes, é possível adicionar tipagem estática para **parâmetros de entrada** e também para o **retorno** usando o Typescript. Tipar esses valores é essencial para a construção de funções, sobretudo aquelas que são compartilhadas em várias partes do sistema.

Para definir tipagem aos parâmetros, basta adicionar o tipo seguido do caractere ":" para cada item. Já o tipo de retorno é definido após a definição dos atributos e antes da abertura do código da função. O tipo de retorno também segue o padrão de uso do caractere ":" como prefixo.

```
// Função de recebe dois números e retorna outro número
function soma(x: number, y: number): number {
  return x + y;
}
```

```
// Função que recebe dois números e retorna uma string
function multip(x: number, y: number): string {
    return `A multiplicação é ${x / y}`;
}

// Função que recebe um texto e retorna um boolean
function textoGrande(texto: string): boolean {
    return texto.length > 10;
}
```

No código acima, existem 3 funções sendo que todas elas possuem a tipagem estática de parâmetros e de retorno. Esta característica facilita o entendimento da própria função, quais são os dados de entrada e qual é o valor gerado na saída. Se um tipo errado for informado como parâmetro ou como atribuição no retorno, o Typescript irá gerar erro.

```
let valor = soma(10, 20);
let valorErrado = soma(10, "20");

let retornoErrado: string = soma(10, 20);
```

O argumento do tipo 'string' não é atribuível ao parâmetro do tipo 'number'. ts(2345)

Exibir o

Problema

Nenhuma correção rápida

disponível

Portanto, a tipagem serve não apenas como **regra de código**, mas também como uma **documentação** que auxilia os desenvolvedores que precisam usar essas funções. É importante notar que um dev geralmente precisa fazer manutenção em códigos de outros devs, e neste caso a documentação de tipagem é fundamental para o entendimento.

ARROW FUNCTIONS

As funções do tipo **arrow functions** também podem definir tipos para seus parâmetros e retorno, bem como as funções normais. A diferença é que uma arrow function é facilmente associada a uma variável que receberá uma **tipagem de função**.

Isto é, um dos tipos permitidos no Typescript é de funções. A sintaxe usada neste caso é (`<params>`) `=>` `<retorno>`, sendo `<params>` a relação de parâmetros (que podem estar tipados) e `<retorno>` o tipo de dados que será retornado na função.

```
const soma = (x: number, y: number): number => {  
    return x + y;  
}  
  
let valor = soma(10, 20);
```

Neste exemplo, a arrow function recebe dois parâmetros numéricos e retorna um outro número, sendo atribuída a constante soma. Logo, o tipo da constante soma é exatamente esta descrição de parâmetros e retorno.

```
const soma: (x: number, y: number) => number  
let valor = soma(10, 20);
```

Com isso, é possível tipar uma arrow function também de forma explícita antes de sua inicialização. A tipagem de funções é muito útil para a criação de contratos: requisitos de parâmetros e retornos de funções que devem ser implementadas posteriormente.

```
// Contrato
let concatenaStrings: (t1: string, t2: string) => string;

// Implementação
concatenaStrings = (t1: string, t2: string) => {
    return t1 + t2;
};
```

ARRAYS

As **listas** (arrays) são estruturas de dados muito usadas na programação. No Typescript, o tipo que representa um array de determinado tipo é o próprio tipo seguido dos caracteres "[]" (abre e fecha colchete, sem espaço entre eles).

Portanto, para criarmos uma lista de string, o tipo será: `string[]`; já para uma lista de números: `number[]`; ou então para uma lista de qualquer tipo de dado (tupla): `any[]`. Uma maneira alternativa é usando a classe genérica `Array` do Typescript: `Array<string>`, `Array<number>` e `Array<any>`.

```
let textos: string[] = ["Web", "Full", "Stack"];
let numeros: number[] = [1, 2, 3];
let tupla: any[] = [1, "Growdev", true];

let textosArray: Array<string> = [];
let numerosArray: Array<number> = [];
let tuplaArray: Array<any> = [];
```

OPERAÇÕES COM ARRAYS

Os arrays possuem um número grande de operações que podem ser feitas sobre os seus elementos. Na maioria das operações, criamos uma função de callback – geralmente arrow function – que itera (percorre) sobre os itens sem necessidade de criarmos uma estrutura de for.

Map

A operação map cria a **cópia de uma lista** permitindo que uma função seja executada para cada item antes de retorná-lo. Ou seja, é possível criar uma cópia customizada (com modificações, sendo que a lista original não é afetada).

O parâmetro do map é uma outra função (função de callback) que será executada para cada item da lista de forma iterativa, um por vez. Esta função, por sua vez, recebe três parâmetros opcionais: value, index e array. O retorno da função é um valor que possui o tipo do array original. Este valor irá compor a lista de saída do map.

```
let textos: string[] = ["Web", "Full", "Stack"];

let copia: string[] = textos.map((value) => {
  return value + " (copia)";
});

console.log("Resultado: ");
console.log(textos);
console.log(copia);
```

```
Resultado:
[ 'Web', 'Full', 'Stack' ]
[ 'Web (copia)', 'Full (copia)', 'Stack (copia)' ]
```

O parâmetro `value` representa o valor do elemento que está sendo iterado, enquanto `index` é a posição do elemento no array. Por fim, `array` é uma referência ao array original – neste caso, alterações **na referência** acabam impactando também no array original.

Reduce

É comum que seja necessário iterar sobre uma lista para obtermos uma **informação resumida sobre o conjunto**, como a soma de todos os elementos em um array de `number`. A operação para este tipo de tarefa é o `reduce`, que itera um array e retorna um único valor – ao invés de um outro array como é o caso do `map`.

O funcionamento do `reduce` pode ser resumido da seguinte forma: cada iteração gera um valor e realiza uma operação com o valor gerado pelas iterações anteriores, até que um valor final seja obtido ao término do array.

Por isso, a função de callback do `reduce` recebe os parâmetros `previous`, `current`, `index` e `array`. O `previous` é o valor retornado pelas iterações anteriores; já o `current` é o valor do elemento que está sendo iterado. Assim como no `map`, podemos acessar o índice do elemento iterado pelo `index` além a referência ao array original com o parâmetro `array`.

```
let numeros: number[] = [15, 20, 30];

let soma: number = numeros.reduce((previous, current) => {
  return previous + current;
});

console.log("Resultado:");
console.log(numeros);
console.log(soma);
```

```
Resultado:  
[ 15, 20, 30 ]  
65
```

É importante perceber que o primeiro parâmetro (`previous`) busca a soma das iterações anteriores. Na primeira iteração, porém, não há valor anterior. Neste caso, o `reduce` entende que a primeira iteração retorna o valor do primeiro elemento e já pula para a segunda iteração.

Por outro lado, podemos iniciar o `reduce` com valor inicial. O segundo parâmetro do `reduce` (após a função de `callback`) indica o valor que será o primeiro `previous`. Este recurso é importante principalmente para `reducers` onde o valor inicial não é, necessariamente, o primeiro item.

```
let numeros: number[] = [15, 20, 30];  
  
let soma: number = numeros.reduce((previous, current) => {  
    return previous + current;  
}, 1000);  
  
console.log("Resultado:");  
console.log(numeros);  
console.log(soma);
```

```
Resultado:  
[ 15, 20, 30 ]  
1065
```

Sort

A função `sort` realiza uma **ordenação no array** baseado em regras definidas de modo iterativo. Nesta operação, a lista original é alterada pois as mudanças são feitas em sua referência.

A função de callback do `sort` recebe dois parâmetros: `previous` e `current`. O parâmetro `current` se refere ao item que está sendo iterado no momento, enquanto `previous` é o item imediatamente anterior a `current` na lista. O retorno do callback é um número.

Se o valor de `previous` é menor do que o de `current`, a função de callback deve retornar um número negativo (em geral, `-1`). Se os valores forem iguais, o retorno deve ser `0`. Já se `previous` é maior do que `current`, o retorno deve ser um valor positivo (em geral, `1`).

```
let numeros: number[] = [99, -10, 20, 40, 30];

console.log("Antes da ordenação: ");
console.log(numeros);

numeros.sort((previous, current) => {
  if (previous > current) return 1;
  if (previous < current) return -1;
  return 0;
});

console.log("Depois da ordenação: ");
console.log(numeros);
```

```
Antes da ordenação:  
[ 99, -10, 20, 40, 30 ]  
Depois da ordenação:  
[ -10, 20, 30, 40, 99 ]
```

Para inverter a ordenação (por exemplo, do maior para o menor), basta inverter a lógica de retorno: retornar negativo quando `current` for menor do que `previous` e vice versa. Além disso, ordenações mais customizadas também podem ser feitas respeitando a regra dos retornos.

Some

A operação `some` **verifica** se determinado item **está presente** no array, retornando um valor booleano (`true` ou `false`). Sua função de callback recebe três parâmetros: `value`, `index` e `array`. Sendo `value` o valor iterado, `index` o índice do elemento e `array` a referência do array original.

Para que um elemento seja considerado encontrado e retorne `true` à operação `some`, a função de callback deve retornar um valor que remete à verdadeiro (não nulo)¹. Isto é pode-se retornar `true` ou mesmo o próprio elemento.

```
let numeros: number[] = [99, -10, 20, 40, 30];  
  
let existe20: boolean = numeros.some((value) => value === 20);  
let existe100: boolean = numeros.some((value) => value === 100);  
  
console.log("Existe algum número 20?");  
console.log(existe20);  
  
console.log("Existe algum número 100?");  
console.log(existe100);
```

¹ No Javascript, um valor verdadeiro pode ser obtido tanto com o boolean `true` quanto com um valor válido qualquer. Já um valor falso é obtido com `false`, `undefined` ou `null`.

```
Existe algum numero 20?  
true  
Existe algum numero 100?  
false
```

Slice

O comando slice retorna **uma cópia de parte do array** original, sendo muito útil quando não precisamos armazenar toda a lista. O slice não possui função de callback, de modo que seus parâmetros são dois números: start e end.

O primeiro parâmetro é o número do índice do elemento inicial para a cópia; já o segundo parâmetro é o índice final. O elemento do índice final não entra na cópia feita pelo slice. Ou seja, a cópia faz o corte a partir do elemento start até o anterior ao end.

```
let numeros: number[] = [99, -10, 20, 40, 30];  
let copia = numeros.slice(1, 4);  
  
console.log("Array original: ");  
console.log(numeros);  
  
console.log("Cópia com slice: ");  
console.log(copia);
```

```
Array original:  
[ 99, -10, 20, 40, 30 ]  
Cópia com slice:  
[ -10, 20, 40 ]
```

Splice

O comando splice é muito interessante e serve para **remover um item** ou um conjunto de itens do array. Este recurso altera a referência do array original, mas também retorna uma cópia dos itens removidos.

O primeiro parâmetro é o `start` que define o item inicial a ser removido. Já o segundo parâmetro – `deleteCount` – é opcional e informa quantos itens do array devem ser removidos a partir de `start`. Se `deleteCount` não for informado, o splice irá remover todos os itens a partir de `start`.

```
let numeros: number[] = [99, -10, 20, 40, 30];
let deletados: number[] = numeros.splice(1, 2);

console.log("Array: ");
console.log(numeros);

console.log("Cópia dos removidos: ");
console.log(deletados);
```

```
Array:
[ 99, 40, 30 ]
Cópia dos removidos:
[ -10, 20 ]
```

Find

O método `find` **busca** e retorna o primeiro **elemento encontrado** pela função de callback. Seu funcionamento é parecido com o da operação `some`, com a diferença de retornar o valor encontrado ao invés de apenas `true` ou `false`. Se o elemento não for encontrado, será retornado `undefined`. Se mais de um item com o mesmo valor for encontrado, **apenas o primeiro** é considerado.

`Find` recebe apenas a função de callback como parâmetro, de modo que esta função recebe `value`, `index` e `array` como parâmetros. Sendo `value` o valor iterado, `index` o índice do elemento e `array` a referência do array original.

```
let numeros: number[] = [99, -10, 20, 40, 30];

let numero20 = numeros.find((value) => value === 20);
let numero100 = numeros.find((value) => value === 100);

console.log("Numero 20 no array: ");
console.log(numero20);
console.log("Numero 100 no array: ");
console.log(numero100);
```

```
Numero 20 no array:
20
Numero 100 no array:
undefined
```

Uma variação deste comando é a função **findIndex**, que funciona exatamente da mesma forma com a exceção de **retornar o índice** que o elemento encontrado possui no array. Neste caso, o retorno é sempre um número que representa o índice.

```
let numeros: number[] = [99, -10, 20, 40, 30];

let numero20 = numeros.findIndex((value) => value === 20);
let numero100 = numeros.findIndex((value) => value === 100);

console.log("Índice do numero 20 no array: ");
console.log(numero20);
console.log("Índice do numero 100 no array: ");
console.log(numero100);
```

Filter

O método filter é semelhante ao find e permite fazer uma **busca** na lista baseada em uma condição na função de callback. A diferença é que o filter retorna **todos os elementos** que atendem a condição – logo o tipo do retorno é uma lista – enquanto o find retorna apenas o primeiro valor.

```
let numeros: number[] = [99, -10, 20, 40, 30];

let numeros20 = numeros.filter((value) => value > 20);
let numeros100 = numeros.filter((value) => value > 100);

console.log("Elementos maiores que 20 no array: ");
console.log(numeros20);
console.log("Elementos maiores que 100 no array: ");
console.log(numeros100);
```

```
Elementos maiores que 20 no array:  
[ 99, 40, 30 ]  
Elementos maiores que 100 no array:  
[]
```



Referências

1. [typescript - npm](#)
2. [TypeScript](#)
3. [The starting point for learning TypeScript](#)
4. [The TypeScript Handbook](#)
5. [Como instalar o TypeScript | Blog TreinaWeb](#)
6. [Typescript: uma introdução para você já sair programando](#)
7. [O que é TypeScript? \[Guia para iniciantes\] – Aplicativos e Software – Tecnoblog](#)
8. [Typescript: o que é e quais são as vantagens de usar Typescript?](#)
9. [Introdução ao TypeScript](#)
10. [Why You Should Use Typescript for Your Next Project - Codemotion Magazine](#)
11. [Why You Should Use TypeScript](#)
12. [Why use TypeScript to write the frontend in 2021? | TSH.io](#)
13. [Por que usar Typescript no Front End? | by Gabriel Almeida | Estante Labs | Medium](#)
14. [Orientação a Objetos e TypeScript: Um guia para iniciantes | by Yan Vancelis](#)
15. [Descomplicando a Programação Orientada a Objetos: Classes, Herança e Interfaces](#)
16. [O que é typescript? Aprenda agora sobre tipagem](#)
17. [Tipagem no TypeScript | Blog TreinaWeb](#)
18. [Array - JavaScript | MDN](#)
19. [JavaScript: métodos de Arrays que você precisa conhecer | Blog TreinaWeb](#)
20. [JavaScript - Arrays : métodos](#)
21. [Entendendo Arrays no JavaScript](#)

BONS ESTUDOS

